

Q L-7.1: Explain the JavaFX application structure

=> JavaFX is a Java framework used to create GUI desktop applications. A JavaFX application is mainly organized using Application, Stage, Scene, and layout panes.

1. Application:

Application is the main class of a JavaFX application. We create a class that extends application. It provides the basic application lifecycle.

2. Stage:

Stage represents the main window of the application. It is the top level container where the application interface is displayed.

3. Scene:

Scene contains the UI components that appear inside a stage. A scene is placed inside a stage.

4. Grid Pane / VBox:

These are layout containers used to arrange UI components. Grid pane arranges components in rows and columns useful for forms.

• Role of start():

start is the main entry point of a JavaFX application. JavaFX automatically calls start() after application is launched. We create the scene, UI components and layout inside start(). Finally the scene is placed on the stage and the stage is displayed.

Q1 L-7.2: Design the GridPane layout: loan Amount, Annual Rate, Number of Years.

=> Loan Calculator Layout:

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class LoanCalculator extends Application {
    @Override
    public void start(Stage stage) {
        GridPane grid = new GridPane();
        grid.setPadding(new Insets(20));
```



```
grid.setHgap(10);
```

```
grid.setVgap(10);
```

```
TextField loanAmount = new TextField();
```

```
TextField annualRate = new TextField();
```

```
TextField years = new TextField();
```

```
Button calculate = new Button("Calculate");
```

```
Label monthlyLabel = new Label("Monthly Payment");
```

```
Label totalLabel = new Label("Total Payment:");
```

```
grid.add(new Label("Loan amount: "), 0, 0);
```

```
grid.add(loanAmount, 1, 0);
```

```
grid.add(new Label("Number of years: "), 0, 1);
```

```
grid.add(years, 1, 1);
```

```
grid.addadd(calculate, 1, 3);
```

```
grid.add(monthlyLabel, 0, 4);
```

```
grid.add(totalLabel, 0, 5);
```

```
Scene scene = new Scene(grid, 400, 300);
```

```
stage.setTitle("Loan Calculator");
```

```
stage.setScene(scene);
```

```
stage.show();
```

```
public static void main (String[] args) {
    launch (args);
}
```

☐ L-7.3 Event Handler computing Monthly Installment, Total Payment and Difference.

⇒ JavaFX Event Handler,

The calculate button uses an event handler to calculate the Monthly Installment, Total Payment and Difference.

Amortization Formula:

$$M = P \frac{r(1+r)^n}{(1+r)^n - 1}$$

P = Loan Amount.

r = Monthly interest rate = Annual Rate / 12 / 100

n = Total number of monthly payments = Y * 12

M = Monthly Installment.

Code:

```
calculate.setOnClickListener {
```

```
    try {
```

```
        double principal = Double.parseDouble(loanAmount.getText());
```

```
        double AnnualRate = Double.parseDouble(rate.getText());
```

```
        int yearsValue = Integer.parseInt(years.getText());
```

```
        double r = AnnualRate / 12 / 100;
```

```
        int n = yearsValue * 12;
```

```
        double monthly;
```

```
        if (r == 0) {
```

```
            monthly = principal / n;
```

```
        } else {
```

```
            monthly = principal * (r * Math.pow(1 + r, n)) /  
                (Math.pow(1 + r, n) - 1);
```

```
        }
```

```
        double TotalPayment = monthly * n;
```

```
        double difference = totalPayment - principal;
```

```
        monthlyLabel.setText(String.format(  
            "Monthly Installment: %.2f", totalPayment));
```

```
differenceLabel.setText(String.format("
Difference : %.2f", difference));
```

```
> catch (NumberFormatException ex) {
```

```
monthlyLabel.setText("Please enter valid
```

```
values: ");
```

```
>
```

```
>);
```

Ex L-8.1 → Compare Socket Programming vs
Java RMI . When to prefer each

Basin	Socket Programming	Java RMI
Communication	Direct communication using sockets	Call method on a remote Java object.
Programm- ing level	Low level	High level
Data exchange	Programmer handles stream and managers.	RMI handles serial- ization and communication.
Complexity	More code and manual handling	Simpler for Java to Java application
Flexibility	More flexible, can communicate with different systems	Mainly suitable for Java applicatio- on.

When to prefer socket Programming :

- When we need direct control over network communication.
- Chat application, custom client server socket real time communication.

When to prefer Java RMI

- When both client and server Java application.
- Suitable distributed Java application where objects need to call method.
- Handle network communication and data serialization.

▣ L → 8.2 Server code using ServerSocket:
accept connection, read via BufferedReader.
reply via PrintWriter:

=>

```
import java.io.*;  
import java.net.*;  
public class Server {  
    public static void main(String[] args) {  
        try {  
            ServerSocket serverSocket = new ServerSocket(8080);  
            System.out.println("Server is waiting");  
            Socket socket = serverSocket.accept();  
            System.out.println("Client Connected");  
            BufferedReader in = new BufferedReader(  
                new InputStreamReader(socket.getInputStream()));  
            PrintWriter out = new PrintWriter(socket.  
                getOutputStream(), true);  
            String message = in.readLine();  
            System.out.println("Client: " + message);  
        }  
    }  
}
```



```

    System.out.println("Hello from Server");
    socket.close();
    serverSocket.close();
} catch (IOException) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

Ex 1 -> 8.3 :

Java Socket Client :

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main (String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            Scanner sc = new Scanner(System.in);

```

```
System.out.print("Enter Manage, ");
```

```
String manage = sc.nextLine();
```

```
PrintWriter out = new PrintWriter(Socket  
    .getOutputStream(), true);
```

```
System.out.println(manage);
```

```
BufferedReader in = new BufferedReader(new  
    InputStreamReader(Socket.getInputStream()));
```

```
String reply = in.readLine();
```

```
System.out.println("Server: " + reply);
```

```
Socket.close();
```

```
} catch (IOException e) {
```

```
    System.out.println("Error: " + e.getMessage());
```

